



NEWTOO

Capacitação JavaScript para Produto, Automação e IA

O que o n8n faz muito bem

- Orquestrar sistemas (APIs, Webhooks, CRMs, bancos)
- Reagir a eventos (gatilhos)
- Encadear ações
- Automatizar tarefas repetitivas
- Fluxos lineares ou semi-lineares
- Integrações rápidas

Ele conecta, coordena e dispara, mas não executa lógica complexa profunda sozinho.

É aí que entra o CODE

- Transformar dados
- Normalizar informação
- Criar regras condicionais mais finas
- Fazer cálculos
- Ajustar formatos
- Tomar decisões intermediárias



Tipos clássicos de código no n8n

1. Enriquecimento simples de dados

Quando você precisa **adicionar ou ajustar campos**.

```
for (const item of $input.all()) {  
    item.json.myNewField = 1;  
}  
  
return $input.all();
```

Use quando:

- O dado não vem pronto da origem
- Você precisa padronizar rapidamente
- Não vale criar API só para isso

2. Normalização de dados

Código que **limpa, valida e padroniza**.

```
return items.map(item => {  
  
    let telefone = item.json.Telefone;
```



```
if (!telefone) {  
    item.json.Telefone_normalizado = null;  
    return item;  
}  
  
telefone = telefone.toString().replace(/\D/g, '');  
  
if (telefone.startsWith('55')) {  
    telefone = telefone.slice(2);  
}  
  
if (telefone.length === 11) {  
    telefone = telefone.slice(0, 2) + telefone.slice(3);  
}  
  
item.json.Telefone_normalizado = `55${telefone}`;  
  
return item;  
});
```

Use quando:

- A fonte é inconsistente
- O destino exige padrão rígido
- A regra é clara e curta

3. Janelas de tempo e lógica condicional

n8n sofre para expressar isso visualmente. Código resolve.

```
const now = new Date();

const output = [];

for (const item of $input.all()) {

  const lead = item.json;

  lead.disparar_followup_dia = false;

  if (

    !lead["Reunião agendada"] ||

    !lead["Data da Reunião"] ||

    lead["Follow-UP Dia"] === true

  ) {

    continue;

  }

  const meetingDate = new Date(lead["Data da Reunião"]);

  if (isNaN(meetingDate.getTime())) continue;

  const diffMinutes =

    (meetingDate.getTime() - now.getTime()) / 60000;

  if (diffMinutes <= 720 && diffMinutes >= 120) {
```



```
    lead.disparar_followup_dia = true;

    lead["Follow-UP Dia"] = true;

    output.push({ json: lead });

  }

}

return output;
```

Use quando:

- Há janelas temporais
- Existem múltiplas condições combinadas
- A lógica começa a ficar ilegível no visual

4. Cálculos

```
const data = items.map(item => item.json);

let maiorGrupo = 0;

for (const item of data) {

  const g = Number(item.Grupo);

  if (!isNaN(g) && g > maiorGrupo) {

    maiorGrupo = g;

  }

}
```



```
}  
  
const proximoGrupo = maiorGrupo + 1;  
  
return [  
  {  
    json: {  
      Grupo_Anterior: maiorGrupo,  
      Proximo_Grupo: proximoGrupo  
    }  
  }  
];
```

Use quando:

- Precisa olhar o conjunto inteiro
- A regra é matemática ou sequencial

5. Manipulação de datas

```
return $input.all().map(item => {  
  const submittedAt = item.json.submittedAt;  
  if (!submittedAt) {  
    item.json.data_submissao_mais_1_dia = null;
```

```
    return item;
}

const date = new Date(submittedAt);
date.setDate(date.getDate() + 1);
const formattedDate = date.toLocaleDateString('pt-BR', {
  day: '2-digit',
  month: 'long',
  year: 'numeric'
});
item.json.data_submissao_mais_1_dia = formattedDate;
return item;
});
```

Use quando:

- Datas precisam ser humanas
- Há soma/subtração de tempo
- O destino não aceita timestamp cru

Quando o n8n começa a NÃO SER suficiente

- Muitos Code Nodes encadeados
- Lógica espalhada em vários fluxos



- Regras difíceis de explicar em português
- Debug doloroso
- Performance começa a importar
- Segurança é crítica
- Versionamento vira problema
- Reuso entre projetos

Quando isso acontece, você não tem mais um fluxo. Você tem uma **aplicação disfarçada**.

Quando criar uma aplicação (Next / Backend)

Crie uma aplicação quando:

- A lógica é o diferencial
- Existem múltiplos usuários
- Há estado complexo
- Banco de dados é central
- A regra muda com frequência
- O sistema precisa escalar
- Você quer testes, logs, versionamento real

1. A lógica é o essencial:

Criamos uma **API dedicada** para:

- enviar mensagens



- criar grupos
- manipular itens do WhatsApp
- padronizar chamadas

Por quê?

- A lógica não era “enviar mensagem”
- Era **como, quando, por qual número, com qual regra**

2. Existem múltiplos usuários

Quando:

- dados não podem se misturar
- regras variam por contexto

Exemplo prático

Cada usuário com:

- números próprios
- regras próprias
- limites próprios

Isso exige:

- autenticação
- autorização
- isolamento lógico

3. Há estado complexo

Estado não é dado solto.

Estado é **histórico + contexto + decisão futura**.

Exemplos de estado complexo:

- “já foi enviado?”
- “em qual etapa está?”
- “qual número foi usado antes?”
- “qual proxy está ativo?”

Exemplo real: gerenciamento de múltiplos números

Criamos uma aplicação para:

- controlar múltiplos números
- associar proxies
- balancear uso
- evitar bloqueios

Isso exige:

- memória
- regras cumulativas
- controle de ciclo de vida

Fluxo não guarda estado.

Aplicação guarda.

4. Banco de dados é central (não acessório)

Quando o banco deixa de ser:

“onde eu salvo algo no final”

e passa a ser:

“onde eu decido o que fazer”

Exemplo real: aplicação de relatórios

Criamos uma aplicação porque:

- o banco não respondia às perguntas certas
- os dados brutos não geravam insight
- era preciso cruzar, filtrar, interpretar

Relatório bom:

- cria visão
- não só exibe tabela

5. A regra muda com frequência

Regras voláteis + fluxo visual = caos.

Se toda semana você precisa:

- alterar condição
- ajustar cálculo



- criar exceção
- versionar comportamento

6. Você quer testes, logs e versionamento real

Quando você precisa:

- log estruturado
- teste automatizado
- auditoria
- observabilidade

0 que vem antes do código

- Input
- Processamento
- Output
- Critério de sucesso

Modelos mentais

- Algoritmo = receita de bolo
- Função = contrato de execução
- Variável = estado do mundo em um ponto do tempo
- Condição = tomada de decisão
- Loop = escala



Exercício-base

Se eu tivesse que explicar essa regra de negócio para uma criança em no máximo 6 passos, eu conseguiria?

Se não → ainda não está pronto para virar código.

Lógica em JavaScript

Tipos fundamentais

- `string` → texto
- `number` → números
- `boolean` → verdadeiro / falso
- `array` → listas
- `object` → entidades (estado organizado)

Estruturas-chave

- Condições: `if / else`
- Repetições: `for, while`
- Funções: `function`

Se uma função faz mais de **uma coisa**, ela está errada.

Ex.: Dado uma lista de leads, queremos **filtrar quem pode receber contato, normalizar o telefone e retornar apenas os válidos**.

```
function filtrarLeadsValidos(leads) {
```



```
// array para armazenar o resultado final

const leadsValidos = [];

// percorre a lista de leads

for (let i = 0; i < leads.length; i++) {

    const lead = leads[i];

    // variáveis (estado do lead)

    let telefone = lead.telefone;

    const ativo = lead.ativo;

    // condição: só continua se o lead estiver ativo e tiver
telefone

    if (!ativo || !telefone) {

        continue;

    }

    // normaliza o telefone (remove tudo que não for número)

    telefone = telefone.toString().replace(/\D/g, '');

    // garante padrão com código do país

    if (!telefone.startsWith('55')) {

        telefone = '55' + telefone;

    }

}
```

```
}

// cria um novo objeto (entidade organizada)

const leadTratado = {

  nome: lead.nome,

  telefone,

  origem: lead.origem

};

// adiciona ao array final

leadsValidos.push(leadTratado);

}

// retorna apenas os leads válidos e tratados

return leadsValidos;

}
```

0 que são bibliotecas

Alguém já:

- Pensou o problema
- Errou
- Corrigiu



- Otimizou

Você só reutiliza.

Exemplos comuns no ecossistema

- UI: Tailwind, Shadcn
- Forms: React Hook Form
- Validação: Zod
- Estado: Zustand
- Datas: Dayjs
- Requisições: Axios / Fetch

Como descobrir bibliotecas boas

- GitHub stars + commits recentes
- Documentação clara
- Adoção pela comunidade
- Compatibilidade com Next

v0 (Vercel)

O que o v0 faz muito bem

- Criar interfaces completas
- Gerar componentes reutilizáveis
- Prototipar produtos reais



- Gerar backend e frontend

Quando levar do navegador para o computador

- Integração com banco de dados diferente do Supabase;
- Quando quiser fazer integração com APIs externas
- Quando quiser fazer Debug real

Estrutura de prompt

Pense no prompt como um **briefing de produto**, não como um pedido.

1. Contexto: O que aconteceu

Aqui você situa a IA no **mundo real**.

Explique:

- Onde isso será usado
- Quem usa
- Em que momento do fluxo
- O que já existe antes

Exemplo de mentalidade: “Estamos criando uma plataforma interna usada por times de vendas para gerenciar envios de WhatsApp.”

Regra: Se a IA não entende o cenário, ela inventa.



2. Problema: O que isso gerou

Mostre a **dor**, não a solução.

Explique:

- O que está travando
- O que é ineficiente hoje
- O que não escala
- O que está errado no fluxo atual

Exemplo: “Hoje cada envio é manual, não temos visão consolidada e o controle de números e status é frágil.”

Regra: Problema mal definido = solução genérica.

3. Objetivo: O que eu quero construir

Aqui você define o **resultado final**, não a implementação.

Explique:

- O que deve existir quando terminar
- Quem usa
- Qual ganho esperado

Exemplo: “Quero uma interface para gerenciar envios, visualizar status, erros e histórico por número.”



4. Operação: Como isso deve funcionar

Descreva:

- Fluxo passo a passo
- Estados possíveis
- Ações do usuário
- Regras básicas (se possível, com a sentença)

Exemplo: “O usuário seleciona um número, escolhe um template, agenda o envio e acompanha o status em tempo real.”

- Dica prática:
Use bullets e sequência lógica.
IA entende fluxo melhor que texto corrido.

5. Referência: Como isso deve parecer

IA cria melhor quando **imita algo existente**.

Inclua:

- Produtos similares
- Padrões visuais conhecidos
- Dashboards reais
- Layouts que funcionam



Exemplo: “A interface deve se inspirar em dashboards do Notion e Vercel, com foco em clareza e leitura rápida.” (coloca um anexo)

*Regra de ouro: IA é excelente em **recombinar**, não em inventar do zero.*

6. Integração – Como as partes conversam

Aqui você já começa a preparar a saída do v0 para o mundo real.

Explique:

- De onde vêm os dados
- Para onde vão
- Se é API, banco, mock
- O que é placeholder

7. Adicionais: Tudo que reduz ambiguidade

Quanto mais contexto, melhor o output.

Inclua sempre que possível:

- Prints
- PDFs
- Fluxos
- Tabelas
- Esquemas
- Dados de exemplo

Exemplo:



“Anexo um print do fluxo atual e um exemplo de payload da API.”

Prompt modelo (framework reutilizável)

Contexto:

Problema:

Objetivo:

Operação:

Referência:

Integração:

Adicionais:

Terminal (CMD)

Comandos essenciais

- `mkdir` → criar pastas
- `cd` → navegar
- `ls / dir` → listar arquivos
- `rm -rf` → apagar
- `git clone` → pegar projetos
- `npm install` → instalar dependências



- `npm run dev` → rodar Next
- `npm run build` → buildar

Exercícios:

NÍVEL 1

1. Soma simples

Crie um programa que receba dois números e retorne a soma.

2. Par ou ímpar

Verifique se um número informado é par ou ímpar.

3. Maior número

Receba dois números e informe qual é o maior.

4. Média de 3 números

Calcule a média de três valores informados.

5. Conversor de temperatura

Converta graus Celsius para Fahrenheit.

6. Antecessor e sucessor

Dado um número, mostre seu antecessor e sucessor.



7. Área do retângulo

Receba base e altura e calcule a área.

8. Velocidade média

Receba distância e tempo e calcule a velocidade média.

9. Nome e idade

Mostre se a pessoa é maior ou menor de idade.

10. Desconto simples

Aplique 10% de desconto em um valor informado.

NÍVEL 2

11. Contagem regressiva

Mostre os números de 10 até 1 usando um `for`.

12. Tabuada

Mostre a tabuada de um número informado (1 a 10).

13. Soma dos pares

Some todos os números pares de 1 a 100.

14. Fatorial

Calcule o fatorial de um número.



15. **Números múltiplos de 3**

Mostre todos os múltiplos de 3 entre 1 e 50.

16. **Validação de senha**

Solicite uma senha até que ela seja correta.

17. **Classificação de nota**

Receba uma nota (0-10) e classifique:

- Reprovado
- Recuperação
- Aprovado

18. **Contador de vogais**

Conte quantas vogais existem em uma palavra.

19. **Ano bissexto**

Verifique se um ano é bissexto.

20. **Número primo**

Verifique se um número é primo.

NÍVEL 3



21. **Maior valor do array**

Dado um array de números, encontre o maior.

22. **Média do array**

Calcule a média dos valores de um array.

23. **Inverter array**

Inverta a ordem dos elementos de um array sem usar `.reverse()`.

24. **Contar ocorrências**

Conte quantas vezes um número aparece no array.

25. **Função soma**

Crie uma função que receba um array e retorne a soma dos elementos.

NÍVEL 4

26. **Remover duplicados**

Dado um array, retorne um novo sem valores duplicados.

27. **Palíndromo**

Verifique se uma palavra é um palíndromo.

28. **Ordenação manual**

Ordene um array sem usar `.sort()`.



29. **Simulador de caixa eletrônico**

Receba um valor e informe quantas notas de 100, 50, 20, 10, 5 e 1 são necessárias.

30. **Validador de CPF (lógica)**

Implemente a lógica de validação de um CPF (sem bibliotecas).

